

## **ĐỒ ÁN CƠ SỞ**

### **ĐỀ TÀI:**

### **INTELLIGENT LOAD BALANCER**

Ngành: **CÔNG NGHỆ THÔNG TIN**  
Chuyên ngành: **MẠNG MÁY TÍNH**

Giáo viên hướng dẫn: ThS. Hàn Minh Châu

Sinh viên thực hiện:

1/ Nguyễn Quang Huy                      MSSV: 2380614932                      Lớp: 23DTHA4

2/ Đoàn Trọng Nghĩa                      MSSV: 2380601439                      Lớp: 23DTHA4

## **ĐỒ ÁN CƠ SỞ**

### **ĐỀ TÀI:** **INTELLIGENT LOAD BALANCER**

Ngành: **CÔNG NGHỆ THÔNG TIN**  
Chuyên ngành: **MẠNG MÁY TÍNH**

Giáo viên hướng dẫn: ThS. Hàn Minh Châu

Sinh viên thực hiện:

|                     |                  |              |
|---------------------|------------------|--------------|
| 1/ Nguyễn Quang Huy | MSSV: 2380614932 | Lớp: 23DTHA4 |
| 2/ Đoàn Trọng Nghĩa | MSSV: 2380601439 | Lớp: 23DTHA4 |

## LỜI CAM ĐOAN

Nhóm chúng em xin cam đoan đề tài **“Intelligent Load Balancer”** là kết quả nhóm tự tìm hiểu, tự thực hiện cấu hình và tự triển khai trong quá trình làm đồ án cơ sở. Các nội dung trình bày trong báo cáo được xây dựng dựa trên quá trình thực hiện đồ án, tham khảo tài liệu và thực nghiệm trực tiếp trên môi trường AWS.

Nhóm có sử dụng một số tài liệu tham khảo liên quan đến các dịch vụ AWS như EC2, Application Load Balancer, Target Group, Health Check, CloudWatch, Auto Scaling và Amazon S3. Những nội dung tham khảo được nhóm dùng để hiểu rõ hơn về lý thuyết và cách triển khai, không sao chép nguyên văn nội dung từ nguồn khác.

Các hình ảnh cấu hình, kết quả kiểm thử và minh chứng trong báo cáo được nhóm chụp lại trong quá trình triển khai hệ thống. Nếu có sai sót trong nội dung báo cáo, nhóm xin chịu trách nhiệm và mong nhận được sự góp ý từ Thầy để hoàn thiện tốt hơn cho đồ án chuyên ngành sắp tới.

TP. Hồ Chí Minh, tháng 05 năm 2026

**Nhóm sinh viên thực hiện**

Nguyễn Quang Huy - Đoàn Trọng Nghĩa

## LỜI CẢM ƠN

Trong quá trình thực hiện và hoàn thiện dự án “**Intelligent Load Balancer**” nhóm muốn gửi lời cảm ơn đến Thầy ThS. Hàn Minh Châu, Khoa Công nghệ Thông Tin Trường Đại học Công nghệ TP.HCM đã tạo điều kiện thuận lợi về cơ sở vật chất, tài liệu học tập và môi trường học tập tích cực đã tận tình hướng dẫn gợi ý, truyền đạt kiến thức chuyên môn về đề tài cũng như những góp ý trong suốt quá trình thực hiện đề tài. Những ý kiến đóng góp từ kinh nghiệm quý báu của các Thầy là nền tảng quan trọng giúp nhóm hoàn thiện dự án này một cách tốt nhất.

Mặc dù nhóm đã nỗ lực tìm hiểu thực hành và hoàn thiện dự án, vì đây là lần đầu tiên tiếp cận thực tế một hệ thống mạng phức tạp, kiến thức chuyên môn và kinh nghiệm thực tế còn hạn chế, sản phẩm và báo cáo của nhóm chắc chắn vẫn còn những điểm chưa được tối ưu. Chúng em rất mong nhận được những lời nhận xét, đánh giá và chỉ bảo thêm từ hội đồng để nhóm có thể nhận ra thiếu sót, rút kinh nghiệm và hoàn thiện kiến thức cho chặng đường học tập sắp tới.

Nhóm em chúc Thầy ThS. Hàn Minh Châu sức khỏe, công tác tốt và luôn thành công trong sự nghiệp giảng dạy.

Nhóm em xin cảm ơn!

TP. Hồ Chí Minh, tháng 05 năm 2026

**Nhóm sinh viên thực hiện**

Nguyễn Quang Huy - Đoàn Trọng Nghĩa

## DANH MỤC VIẾT TẮT

| Viết tắt | Nghĩa đầy đủ                       |
|----------|------------------------------------|
| AWS      | Amazon Web Services                |
| ALB      | Application Load Balancer          |
| EC2      | Elastic compute Cloud              |
| ELB      | Elastic load Balancing             |
| ASG      | Auto Scaling Group                 |
| TG       | Target Group                       |
| CPU      | Central Processing Unit            |
| RAM      | Random access memory               |
| HTTPS    | HyperText Transfer Protocol Secure |
| HTTP     | Hyper Text Transfer Protocol       |
| S3       | Simple Storage Service             |
| CLI      | Command Line Interface             |
| IP       | Internet Protocol                  |
| DNS      | Domain Name System                 |
| SSH      | Secure Shell                       |
| AMI      | Amazon Machine Image               |

# MỤC LỤC

|                                                             |      |
|-------------------------------------------------------------|------|
| LỜI CAM ĐOAN.....                                           | iii  |
| LỜI CẢM ƠN .....                                            | iv   |
| DANH MỤC VIẾT TẮT .....                                     | v    |
| DANH MỤC HÌNH .....                                         | viii |
| CHƯƠNG 1: TỔNG QUAN.....                                    | 1    |
| 1.1 Lý do chọn đề tài .....                                 | 1    |
| 1.2 Mục tiêu đề tài .....                                   | 1    |
| 1.3 Đối tượng và phạm vi thực hiện đề tài .....             | 2    |
| 1.4 Nhiệm vụ đề tài.....                                    | 2    |
| CHƯƠNG 2: CƠ SỞ LÝ THUYẾT .....                             | 3    |
| 2.1 Tổng quan về Load Balancer .....                        | 3    |
| 2.2 Các thuật toán cân bằng tải phổ biến.....               | 4    |
| 2.2.1 Round Robin .....                                     | 4    |
| 2.2.2 Least Connection .....                                | 4    |
| 2.2.3 Cân bằng tải dựa trên trạng thái server .....         | 5    |
| 2.3 Tổng quan Amazon Web Services .....                     | 6    |
| 2.4 Amazon EC2.....                                         | 6    |
| 2.5 Elastic Load Balancer và Application Load Balancer..... | 7    |
| 2.5.1 Elastic Load Balancer .....                           | 7    |
| 2.5.2 Application Load Balancer .....                       | 7    |
| 2.6 Target Group.....                                       | 7    |
| 2.7 Health Check .....                                      | 8    |
| 2.8 Auto Scaling .....                                      | 8    |
| 2.9 Cơ chế phân phối request trong mô hình.....             | 9    |
| CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM.....                          | 10   |
| 3.1 Mô hình triển khai hệ thống .....                       | 10   |
| 3.2 Các thành phần trong mô hình triển khai trên AWS .....  | 12   |
| 3.2.1 Application Load Balancer .....                       | 12   |
| 3.2.2 Listener của Application Load Balancer .....          | 12   |

|                                                     |    |
|-----------------------------------------------------|----|
| 3.2.3 Target Group .....                            | 13 |
| 3.2.4 Health Check.....                             | 13 |
| 3.2.5 EC2 Web Server .....                          | 14 |
| 3.2.6 Launch Template .....                         | 14 |
| 3.2.7 Auto Scaling Group .....                      | 15 |
| 3.3 Kiểm tra hoạt động hệ thống .....               | 16 |
| 3.3.1 Truy cập ứng dụng thông qua DNS của ALB ..... | 16 |
| 3.3.2 Kiểm thử phân phối request đến các EC2.....   | 16 |
| 3.4 Giám sát và tự động mở rộng hệ thống .....      | 17 |
| 3.5 Ghi nhận Access Log và Dashboard giám sát ..... | 21 |
| 3.5.1 Access Log trên Amazon S3 .....               | 21 |
| 3.5.2 Nội dung log request.....                     | 21 |
| 3.5.3 Dashboard giám sát hệ thống.....              | 22 |
| CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ .....               | 24 |
| 4.1 Kết luận.....                                   | 24 |
| 4.2 Hạn chế của đề tài.....                         | 24 |
| 4.3 Hướng phát triển.....                           | 24 |
| TÀI LIỆU THAM KHẢO .....                            | 25 |

## DANH MỤC HÌNH

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Hình 2.1: Mô hình tổng quan về load balancer. ....                                    | 3  |
| Hình 2.2: Thuật toán round robin. ....                                                | 4  |
| Hình 2.3: Thuật toán least connection. ....                                           | 5  |
| Hình 2.4: Cân bằng tải dựa trên trạng thái server. ....                               | 5  |
| Hình 3.1: Mô hình tổng quan hệ thống Intelligent Load Balancer. ....                  | 10 |
| Hình 3.2: Application Load Balancer tạo trên AWS. ....                                | 12 |
| Hình 3.3: Listener HTTP port 80 chuyển tiếp request đến Target Group. ....            | 12 |
| Hình 3.4: Các EC2 trong Target Group ở trạng thái Healthy. ....                       | 13 |
| Hình 3.5: Cấu hình Health Check của Target Group. ....                                | 13 |
| Hình 3.6: Danh sách EC2 Web Server đang chạy trong hệ thống. ....                     | 14 |
| Hình 3.7: Launch Template dùng để khởi tạo EC2 cho Auto Scaling Group. ....           | 14 |
| Hình 3.8: Auto Scaling Group hệ thống. ....                                           | 15 |
| Hình 3.9: Truy cập ứng dụng thông qua DNS của Application Load Balancer. ....         | 16 |
| Hình 3.10: Request được phân phối đến EC2 Web Server thứ nhất. ....                   | 16 |
| Hình 3.11: Request được phân phối đến EC2 Web Server thứ hai. ....                    | 17 |
| Hình 3.12: Giả lập ép tăng/ giảm tải CPU trên EC2 Web Server bằng lệnh Terminal. .... | 17 |
| Hình 3.13: CloudWatch giám sát số lượng request của Application Load Balancer. ....   | 18 |
| Hình 3.14: CloudWatch Alarm theo dõi ngưỡng CPU của hệ thống. ....                    | 18 |
| Hình 3.15: Số lượng EC2 thay đổi theo quá trình Auto Scaling. ....                    | 19 |
| Hình 3.16: Cấu hình Capacity của Auto Scaling Group trong hệ thống. ....              | 20 |
| Hình 3.17: Lịch sử Auto Scaling Group tạo mới và thu hồi EC2 instance. ....           | 20 |
| Hình 3.18: Access Log của Application Load Balancer trên Amazon S3. ....              | 21 |
| Hình 3.19: Nội dung Access Log ghi nhận request đi qua Load Balancer. ....            | 21 |
| Hình 3.20: Dashboard liên kết AWS giám sát hệ thống. ....                             | 22 |
| Hình 3.21: Dashboard liên kết AWS giám sát hệ thống khi tăng/ giảm tải. ....          | 23 |



# CHƯƠNG 1: TỔNG QUAN

## 1.1 Lý do chọn đề tài

Các hệ thống web và dịch vụ trực tuyến đã trở thành một phần không thể thiếu từ trong học tập đến làm việc cho đến giải trí và mua sắm. Điều đó đồng nghĩa với việc lượng truy cập vào các hệ thống này ngày càng lớn hơn và khó dự đoán hơn. Khi toàn bộ lưu lượng dồn vào một máy chủ duy nhất mà không có cơ chế phân phối hợp lý, hệ thống rất dễ rơi vào trạng thái quá tải phản hồi chậm, nghẽn cổ chai, thậm chí gián đoạn hoàn toàn vào đúng lúc người dùng cần nhất.

Nhóm lựa chọn đề tài “Intelligent Load Balancer” nhằm tìm hiểu cách xây dựng một mô hình cân bằng tải cho hệ thống web trên môi trường AWS. Ví dụ thực tế, nếu một web chỉ chạy trên một máy chủ, toàn bộ request của người dùng sẽ tập trung vào cùng một điểm xử lý. Khi lượng truy cập tăng cao hoặc server gặp sự cố, hệ thống có thể phản hồi chậm, mất ổn định hoặc bị gián đoạn. Trong mô hình của nhóm thì người dùng không truy cập trực tiếp vào từng EC2 Web Server mà truy cập thông qua Application Load Balancer. Load Balancer sẽ tiếp nhận request và phân phối đến các EC2 phía sau thông qua Target Group. Các server được kiểm tra trạng thái bằng Health Check, nhờ đó hệ thống có thể hạn chế gửi request đến những EC2 không hoạt động bình thường. Nhóm kết hợp thêm CloudWatch và Auto Scaling Group để theo dõi tải và hỗ trợ mở rộng số lượng EC2 khi cần thiết.

## 1.2 Mục tiêu đề tài

Tìm hiểu và triển khai cấu hình mô hình Intelligent Load Balancer cho hệ thống web trên môi trường AWS. Hiểu rõ hơn cách nhiều máy chủ có thể cùng xử lý yêu cầu truy cập từ người dùng, thay vì để toàn bộ lưu lượng dồn vào một server duy nhất. Thực hiện triển khai nhiều máy chủ EC2, cấu hình Target Group, thiết lập Application Load Balancer, kiểm tra trạng thái hoạt động của các server thông qua Health Check và tiến hành kiểm thử để quan sát quá trình phân phối request đến các EC2 phía sau Load

Balancer. Đề tài cũng giúp nhóm làm quen với việc triển khai hệ thống trên môi trường điện toán đám mây, hiểu thêm về khả năng mở rộng với Auto Scaling và vai trò của Load Balancer trong việc giúp hệ thống hoạt động ổn định hơn khi lượng truy cập tăng.

### **1.3 Đối tượng và phạm vi thực hiện đề tài**

Đồ án tập trung nghiên cứu cơ chế hoạt động và cách phân phối lưu lượng của hệ thống Load Balancing, nhóm tìm hiểu các dịch vụ quản lý hạ tầng mạng cốt lõi trên nền tảng AWS. Các thành phần đối tượng chính bao gồm máy chủ ảo EC2, bộ cân bằng tải ứng dụng Application Load Balancer - ALB, Target Group, cùng với các cơ chế giám sát máy chủ Health Check và tính năng co giãn tự động của Auto Scaling.

Đề tài được triển khai trực tiếp trên môi trường Cloud của AWS. Nhóm tập trung vào cân bằng tải tầng ứng dụng cho Web Server sử dụng giao thức HTTPs. Trong mô hình này, Application Load Balancer sẽ phân phối request đến nhiều EC2 Web Server phía sau, giúp giảm áp lực cho từng server và hạn chế phụ thuộc vào một máy chủ duy nhất. Nhóm cũng thực hiện giả lập tải để quan sát quá trình phân phối request và khả năng mở rộng thông qua Auto Scaling. Do giới hạn về thời gian, kinh nghiệm và chi phí sử dụng tài nguyên AWS, đề tài chưa mở rộng sang cân bằng tải cơ sở dữ liệu hoặc các giải pháp bảo mật chuyên sâu.

### **1.4 Nhiệm vụ đề tài**

Trong quá trình thực hiện đề tài, nhóm tìm hiểu cách hoạt động của Load Balancer và các dịch vụ AWS liên quan như EC2, Application Load Balancer, Target Group, Health Check và Auto Scaling Group. Về phần triển khai, nhóm tạo nhiều EC2 instance làm Web Server, cấu hình Target Group và Application Load Balancer để phân phối request. Đồng thời, nhóm sử dụng Health Check để kiểm tra trạng thái EC2 và cấu hình Auto Scaling nhằm hỗ trợ hệ thống mở rộng hoặc thu hẹp khi tải thay đổi.

Sau khi cấu hình xong, nhóm kiểm thử bằng cách truy cập website thông qua DNS của Load Balancer, quan sát kết quả phản hồi từ các EC2 và đánh giá khả năng phân phối traffic của mô hình.

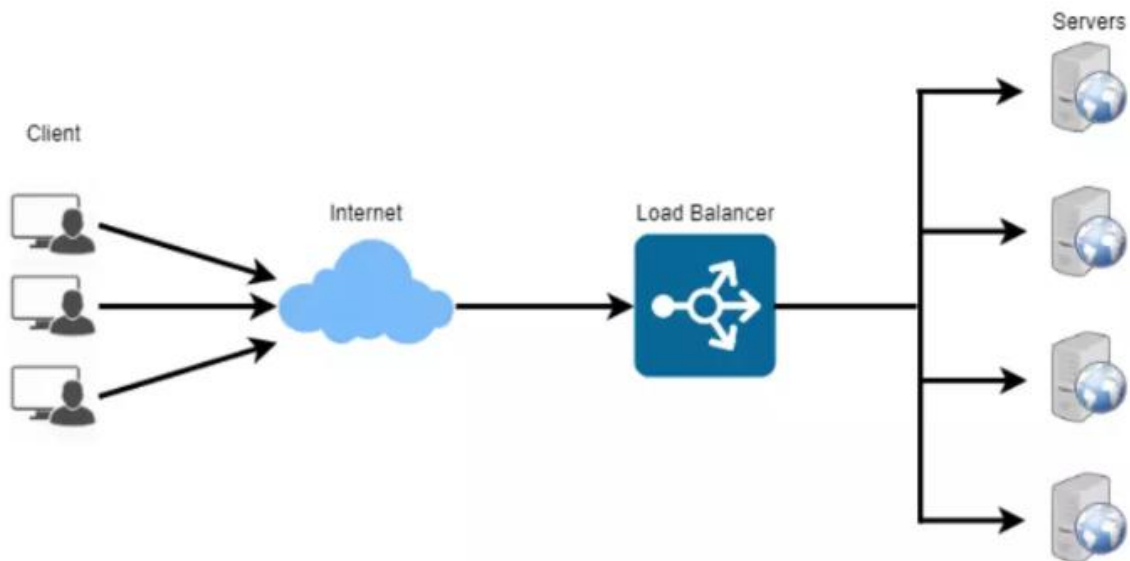
## CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

### 2.1 Tổng quan về Load Balancer

Load Balancer là thành phần trung gian đứng giữa người dùng và các máy chủ phía sau. Khi người dùng gửi yêu cầu truy cập vào hệ thống, request sẽ không đi trực tiếp vào một server cố định mà được chuyển đến Load Balancer trước. Load Balancer sẽ phân phối request đến một trong các server đang hoạt động theo cấu hình trong hệ thống.

Trong hệ thống web, nếu tất cả lưu lượng chỉ tập trung vào một máy chủ thì server rất dễ bị quá tải khi số lượng người dùng tăng. Điều này có thể làm web phản hồi chậm, lỗi truy cập hoặc ngừng hoạt động. Load Balancer được dùng để chia request cho nhiều server, giúp giảm áp lực cho từng máy và hỗ trợ hệ thống hoạt động ổn định hơn. Ngoài việc phân phối lưu lượng, Load Balancer còn giúp hạn chế gián đoạn khi một server gặp lỗi. Nếu server phía sau không phản hồi, Load Balancer có thể ngừng gửi request đến server đó và tiếp tục chuyển request đến các server còn hoạt động bình thường.

Trong đề tài này, nhóm sử dụng Application Load Balancer trên AWS để cân bằng tải cho các EC2 Web Server. ALB hoạt động ở tầng ứng dụng, phù hợp với hệ thống web sử dụng giao thức HTTP/HTTPS.



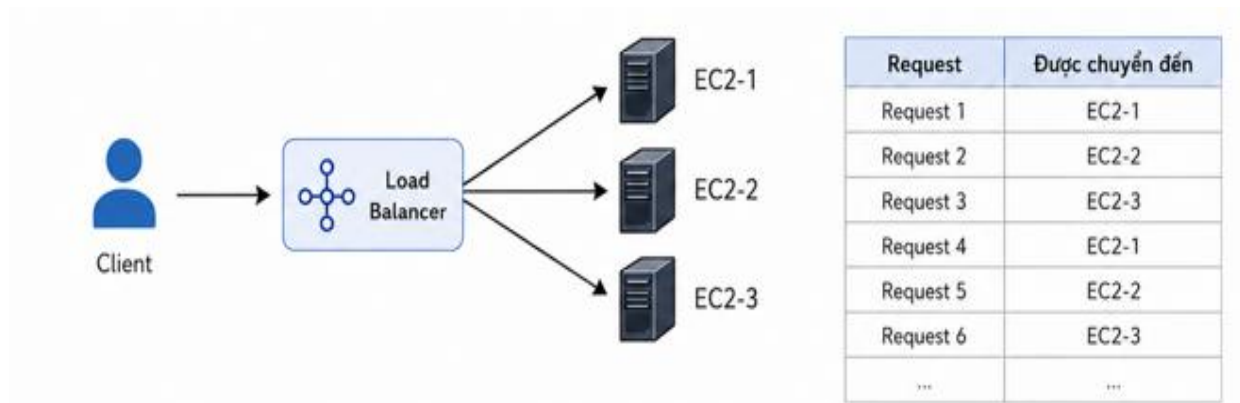
Hình 2.1: Mô hình tổng quan về load balancer.

## 2.2 Các thuật toán cân bằng tải phổ biến

### 2.2.1 Round Robin

Round Robin là thuật toán đơn giản và phổ biến nhất trong các hệ thống cân bằng tải, các request đến sẽ được phân phối lần lượt theo vòng tròn đến từng server trong danh sách server 1 nhận request thứ nhất, server 2 nhận request thứ hai, server 3 nhận request thứ ba, rồi quay lại server 1 cho request tiếp theo và cứ thế tiếp tục.

Ưu điểm lớn nhất của Round Robin là đơn giản, dễ cấu hình và không cần theo dõi trạng thái của server. Tuy nhiên, vấn đề nằm ở chỗ thuật toán này không quan tâm đến việc server đang bận hay rảnh miễn đến lượt là nhận. Trong thực tế, nếu có một server đang gánh một tác vụ nặng mà vẫn tiếp tục nhận thêm request mới, hiệu suất toàn hệ thống sẽ bị ảnh hưởng khi chỉ nhìn bề ngoài nhận thấy sự phân phối khá đều.

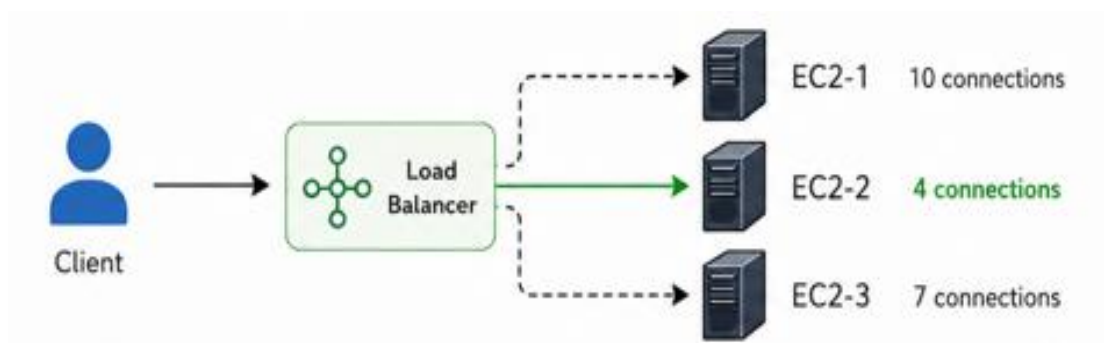


Hình 2.2: Thuật toán round robin.

### 2.2.2 Least Connection

Least Connection ra đời để khắc phục đúng điểm yếu của Round Robin. Thuật toán Least Connection khác ở chỗ là nhìn vào số lượng kết nối đang được xử lý tại từng server rồi chọn cái nào đang nhàn nhất để chuyển request tiếp theo sang.

Cách làm này hợp lý hơn trong các hệ thống mà mỗi request có độ phức tạp khác nhau có request xử lý xong trong vài chục millisecond nhưng cũng có request cần vài giây. Least Connection ít nhất sẽ tránh được việc dồn quá nhiều việc vào cùng một chỗ.

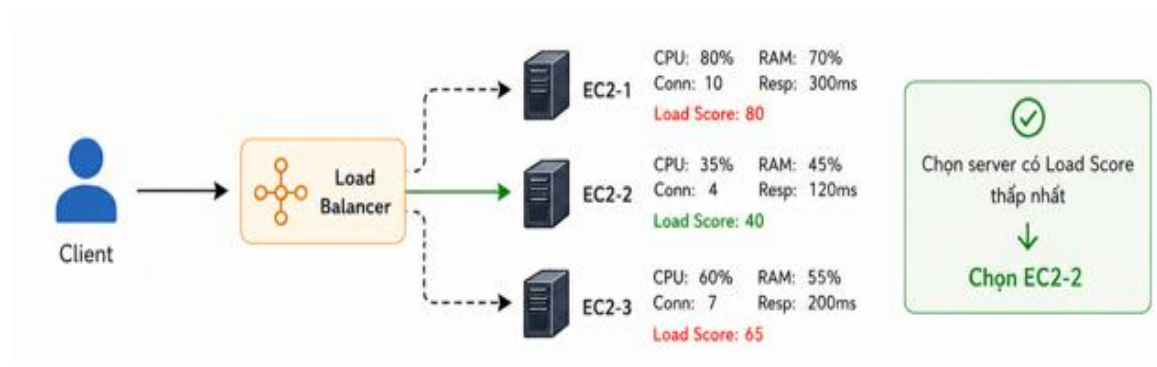


Hình 2.3: Thuật toán least connection.

### 2.2.3 Cân bằng tải dựa trên trạng thái server

Đây là hướng tiếp cận mà nhóm thấy gần với tên đề tài nhất, hệ thống không chỉ đếm kết nối hay phân phối theo vòng, mà thực sự đọc dữ liệu từ server để quyết định. Các chỉ số như mức CPU, lượng RAM còn lại hay độ trễ phản hồi được thu thập liên tục, và dựa vào đó hệ thống mới điều phối request cho phù hợp.

Amazon CloudWatch dùng để giám sát các thông số kỹ thuật, chỉ số đó theo thời gian thực. Khi CPU của một server vượt ngưỡng, CloudWatch Alarm sẽ kích hoạt Scaling Policy để tự động thêm instance mới vào hệ thống thay vì cứ tiếp tục nhồi request vào một chỗ đang bị nghẽn. Đây là điểm khác biệt rõ nhất so với Round Robin hay Least Connection hệ thống phản ứng theo tình huống thực tế chứ không chạy theo một quy tắc cứng nhắc định sẵn.



Hình 2.4: Cân bằng tải dựa trên trạng thái server.

## 2.3 Tổng quan Amazon Web Services

Amazon Web Services là nền tảng về điện toán đám mây của Amazon. Thay vì phải tự mua máy chủ, lắp đặt phòng máy và quản lý phần cứng, người dùng có thể thuê các tài nguyên như máy chủ ảo, lưu trữ, mạng và cơ sở dữ liệu trực tiếp trên AWS. Ưu điểm Amazon Web Services là có thể tăng, giảm tài nguyên theo nhu cầu sử dụng. Khi hệ thống có nhiều người truy cập, người quản trị có thể mở rộng thêm máy chủ. Khi nhu cầu giảm, lượng tài nguyên được thu lại để tiết kiệm tối ưu chi phí kinh tế.

Trong đề án, nhóm sử dụng AWS để triển khai mô hình Intelligent Load Balancer. Các dịch vụ chính được sử dụng gồm EC2 để tạo máy chủ web, Application Load Balancer để phân phối request, Target Group để quản lý các server phía sau, Health Check để kiểm tra trạng thái server và Auto Scaling để hỗ trợ mở rộng hệ thống khi cần thiết. Nhờ triển khai trên AWS, nhóm có một hệ thống web có nhiều server cùng xử lý request, từ đó hiểu rõ hơn cách Load Balancer giúp hệ thống hoạt động ổn định và giảm tình trạng quá tải.

## 2.4 Amazon EC2

Amazon EC2 (Elastic Compute Cloud) là dịch vụ cho phép tạo và vận hành máy chủ ảo trên hạ tầng cloud của AWS. Hình dung đơn giản thì mỗi EC2 instance hoạt động gần giống một chiếc máy tính thông thường có hệ điều hành, có thể cài phần mềm, chạy ứng dụng web nhưng toàn bộ phần cứng phía sau đều do Amazon quản lý, người dùng chỉ cần quan tâm đến phần cấu hình và triển khai bên trên.

Trong đề tài này, nhóm sử dụng nhiều EC2 instance để đóng vai trò các Web Server nằm phía sau Load Balancer. Mỗi instance được cài một ứng dụng web cơ bản để tiếp nhận và xử lý request khi người dùng truy cập vào hệ thống, request sẽ đi qua ALB trước, rồi từ đó mới được chuyển tiếp đến một trong các EC2 đang hoạt động tùy theo chiến lược phân phối đã cấu hình. Lý do nhóm chọn chạy nhiều EC2 thay vì một instance duy nhất cũng khá thực tế nếu một server gặp sự cố hay mất kết nối, Load Balancer sẽ phát hiện thông qua Health Check và tự động dừng chuyển request đến chỗ đó, phần còn lại vẫn tiếp tục chạy bình thường mà người dùng gần như không nhận ra có gì bất ổn.

## **2.5 Elastic Load Balancer và Application Load Balancer**

### **2.5.1 Elastic Load Balancer**

Elastic Load Balancer (ELB) là dịch vụ cân bằng tải của AWS. Dịch vụ này có nhiệm vụ tiếp nhận lưu lượng truy cập từ người dùng, sau đó phân phối các request đến những máy chủ phía sau như EC2. Nhờ vậy, hệ thống không bị phụ thuộc vào một server duy nhất và có thể hoạt động ổn định hơn khi lượng truy cập tăng.

ELB cũng hỗ trợ kiểm tra trạng thái của các máy chủ thông qua cơ chế Health Check. Nếu một EC2 gặp lỗi hoặc không phản hồi, Load Balancer sẽ hạn chế gửi request đến máy chủ đó và tiếp tục chuyển lưu lượng đến các EC2 còn hoạt động bình thường. Đây là điểm quan trọng giúp hệ thống giảm nguy cơ gián đoạn dịch vụ. Trong AWS có nhiều loại Load Balancer khác nhau, nhưng trong đồ án này nhóm tập trung sử dụng Application Load Balancer vì mô hình triển khai chủ yếu xử lý request web thông qua giao thức HTTP.

### **2.5.2 Application Load Balancer**

Application Load Balancer là loại Load Balancer hoạt động ở tầng ứng dụng, thường dùng cho các hệ thống web chạy HTTP hoặc HTTPS. ALB nhận request từ người dùng, sau đó chuyển request đến Target Group chứa các EC2 Web Server phía sau. Trong mô hình của nhóm, ALB đóng vai trò là nơi truy cập và kiểm tra chính của hệ thống. Người dùng không truy cập trực tiếp vào từng EC2 mà truy cập thông qua địa chỉ DNS của ALB. Sau đó, ALB sẽ phân phối request đến các EC2 đang ở trạng thái hoạt động tốt.

Việc sử dụng ALB giúp mô hình dễ quản lý hơn vì các EC2 phía sau có thể được thêm, xóa hoặc thay đổi mà người dùng không cần biết chi tiết. ALB cũng phù hợp với đề tài vì hệ thống của nhóm tập trung vào cân bằng tải cho Web Server ở tầng ứng dụng.

## **2.6 Target Group**

Target Group là nơi tập hợp các máy chủ đích mà Load Balancer sẽ chuyển request đến. Trong AWS thay vì cấu hình trực tiếp từng EC2 vào Load Balancer, người dùng sẽ

gom các server đó vào một Target Group. ALB chỉ cần biết Target Group là gì, còn bên trong có bao nhiêu EC2 và trạng thái ra sao thì Target Group tự quản lý.

Trong mô hình của nhóm, Target Group chứa các EC2 Web Server đã được cài sẵn ứng dụng web để tiếp nhận request. Khi người dùng truy cập vào địa chỉ DNS của ALB, request sẽ đi qua Load Balancer trước, sau đó ALB dựa vào Target Group để xác định nên chuyển request đó đến EC2 nào đang sẵn sàng phục vụ.

Ngoài việc quản lý danh sách EC2, Target Group còn kết hợp với Health Check để kiểm tra trạng thái từng server. EC2 nào phản hồi bình thường sẽ được đánh dấu Healthy và tiếp tục nhận request từ Load Balancer. Khi thay đổi số lượng server phía sau, người dùng vẫn truy cập hệ thống thông qua DNS của ALB, không cần biết cụ thể request đang được xử lý bởi EC2 nào.

## **2.7 Health Check**

Health Check là cơ chế dùng để kiểm tra trạng thái hoạt động của các server phía sau Load Balancer. Trong mô hình AWS, Health Check giúp Application Load Balancer biết EC2 nào đang hoạt động bình thường và EC2 nào đang gặp lỗi hoặc không phản hồi.

Trong đồ án này, Health Check được cấu hình trong Target Group để kiểm tra các EC2 Web Server. Nếu một EC2 phản hồi đúng theo đường dẫn và cổng đã cấu hình, hệ thống sẽ đánh dấu server đó ở trạng thái Healthy. Load Balancer có thể tiếp tục phân phối request đến EC2 này. Ngược lại, nếu một EC2 không phản hồi hoặc phản hồi lỗi trong một khoảng thời gian nhất định, server đó sẽ bị đánh dấu là Unhealthy. Lúc này, Load Balancer sẽ không ưu tiên chuyển request đến EC2 bị lỗi, nhằm tránh làm ảnh hưởng đến người dùng khi truy cập hệ thống.

## **2.8 Auto Scaling**

Auto Scaling là tính năng giúp hệ thống tự động thay đổi số lượng EC2 theo tình trạng tải. Khi lượng truy cập tăng hoặc CPU vượt ngưỡng cấu hình, hệ thống có thể tạo thêm EC2 để chia tải. Ngược lại, khi tải giảm, số lượng EC2 có thể được giảm bớt để tiết kiệm tài nguyên. Trong mô hình của nhóm, Auto Scaling Group được kết hợp với Application Load Balancer và Target Group. Các EC2 được tạo mới sẽ được đưa vào



Target Group để Load Balancer phân phối request đến chúng. Nhờ đó, hệ thống có thể linh hoạt hơn khi lượng truy cập thay đổi.

Trong đồ án này, Auto Scaling được sử dụng để hỗ trợ mô hình cân bằng tải hoạt động ổn định hơn. Tính năng này giúp nhóm hiểu rõ hơn cách một hệ thống web có thể tự mở rộng khi gặp tải cao, thay vì phải thêm máy chủ thủ công như cách triển khai truyền thống.

## **2.9 Cơ chế phân phối request trong mô hình**

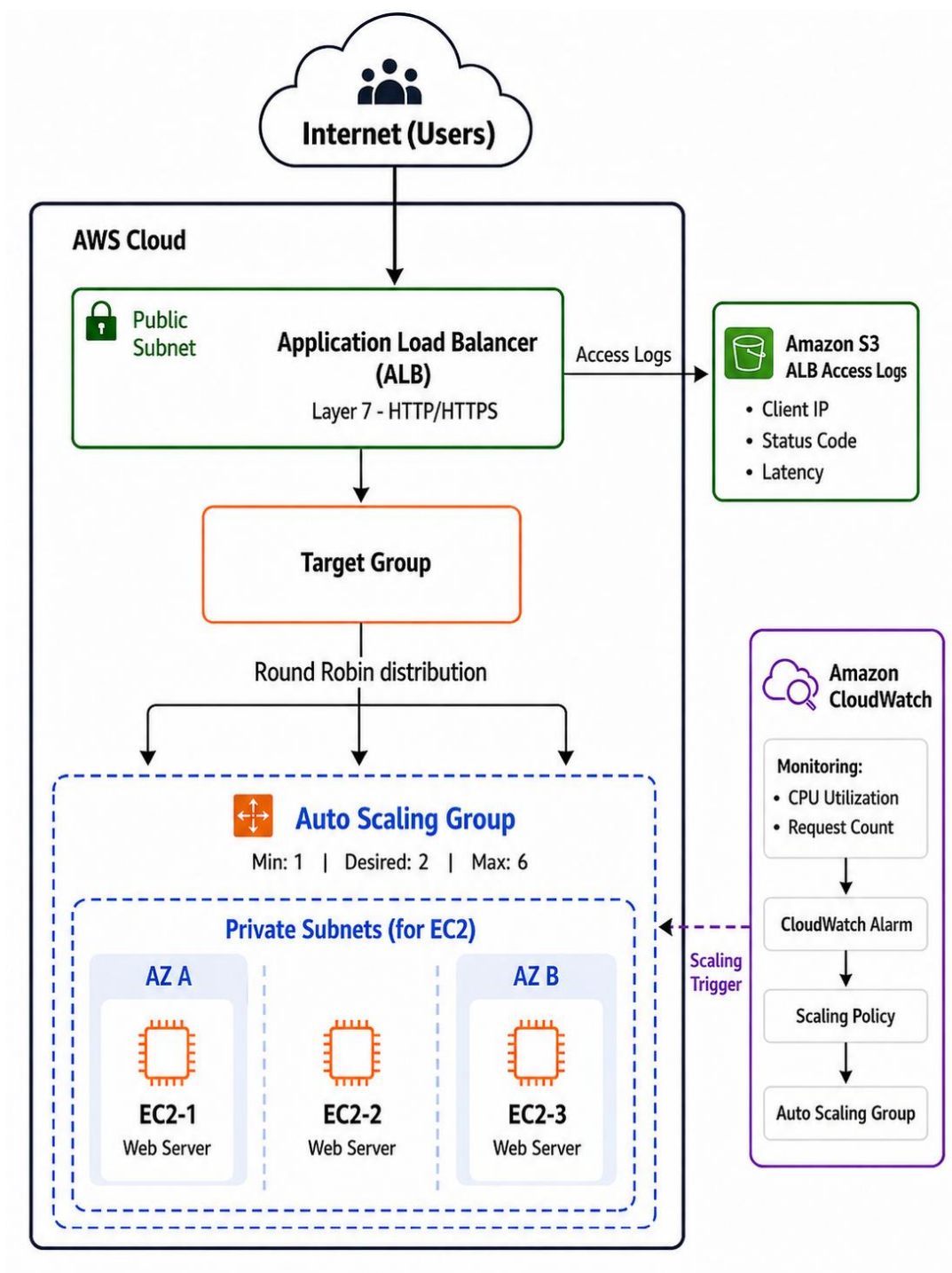
Trong mô hình của nhóm, người dùng không truy cập trực tiếp vào từng máy chủ EC2 riêng lẻ mà sẽ truy cập thông qua địa chỉ DNS của Application Load Balancer. Khi một request được gửi đến hệ thống, ALB sẽ là thành phần tiếp nhận đầu tiên, sau đó chuyển request này đến Target Group chứa các EC2 Web Server phía sau.

Trước khi phân phối request, Load Balancer sẽ dựa vào kết quả Health Check để xác định server nào đang hoạt động ổn định. Những EC2 ở trạng thái Healthy sẽ được nhận request, còn những EC2 bị lỗi hoặc không phản hồi sẽ tạm thời không được ưu tiên nhận lưu lượng. Nhờ vậy, hệ thống có thể hạn chế việc gửi request đến server đang gặp sự cố.

Sau khi chọn được EC2 phù hợp trong Target Group, request sẽ được chuyển đến server đó để xử lý và trả kết quả về cho người dùng. Trong trường hợp lượng truy cập tăng cao, Auto Scaling có thể hỗ trợ tạo thêm EC2 mới. Khi các EC2 mới hoạt động ổn định và vượt qua Health Check, chúng sẽ được đưa vào Target Group để cùng tham gia xử lý request.

## CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM

### 3.1 Mô hình triển khai hệ thống



Hình 3.1: Mô hình tổng quan hệ thống Intelligent Load Balancer.

Mô hình gồm 2 luồng chính:

- Luồng xử lý request chính: User Request → Application Load Balancer → Target Group → EC2 Healthy → Response trả về User.
- Luồng giám sát và tự động mở rộng: EC2 / ALB Metrics → CloudWatch → CloudWatch Alarm → Scaling Policy → Auto Scaling Group → Thêm/Giảm EC2 → Target Group.

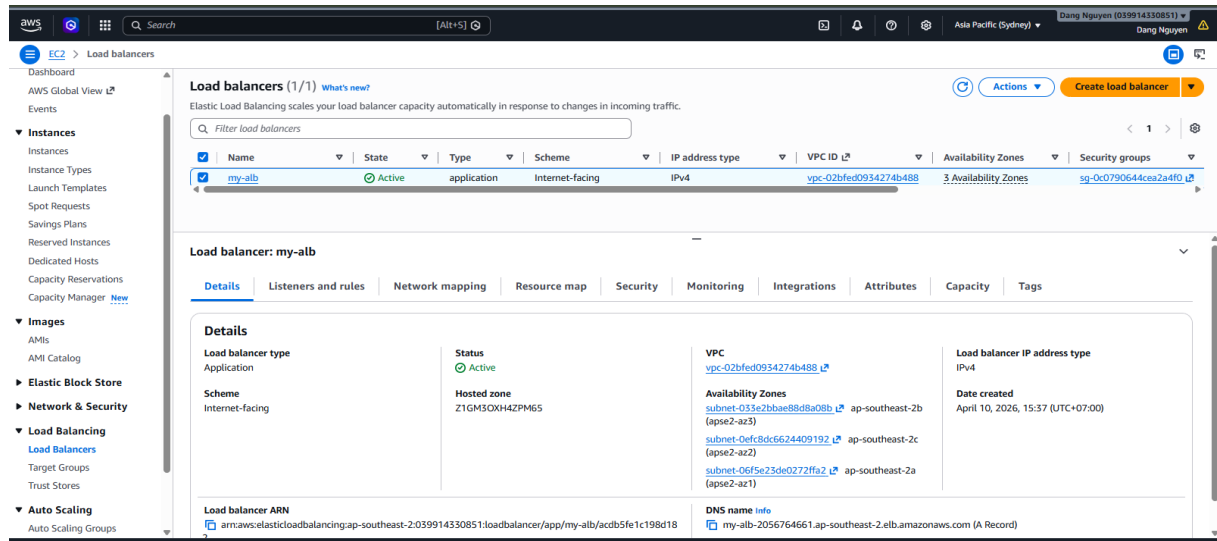
Mô hình Intelligent Load Balancer được nhóm triển khai trên AWS để mô phỏng hệ thống web có nhiều EC2 Web Server cùng xử lý request. Người dùng không truy cập trực tiếp vào từng EC2 mà truy cập thông qua DNS của Application Load Balancer.

Khi có request gửi đến, ALB sẽ tiếp nhận và chuyển request vào Target Group. Các EC2 trong Target Group sẽ được kiểm tra trạng thái bằng Health Check. Những EC2 ở trạng thái Healthy sẽ tiếp tục nhận request, còn EC2 lỗi hoặc không phản hồi sẽ tạm thời không được phân phối lưu lượng. Nhóm sử dụng CloudWatch để theo dõi các chỉ số như CPU Utilization và Request Count. Khi tải vượt ngưỡng cấu hình, CloudWatch Alarm sẽ kích hoạt Scaling Policy để Auto Scaling Group tự động tăng số lượng EC2. Access Log của ALB cũng được lưu vào Amazon S3 để nhóm kiểm tra lại thông tin request trong quá trình thử nghiệm.

Về mặt bảo mật, theo mô hình triển khai thực tế EC2 Web Server nên được đặt trong Private Subnet. Trong phạm vi demo nhóm gắn Public IP tạm thời để thuận tiện SSH và kiểm thử tăng tải. Khi triển khai thực tế, nhóm sẽ đưa EC2 về Private Subnet và sử dụng NAT Gateway để đảm bảo an toàn hơn.

## 3.2 Các thành phần trong mô hình triển khai trên AWS

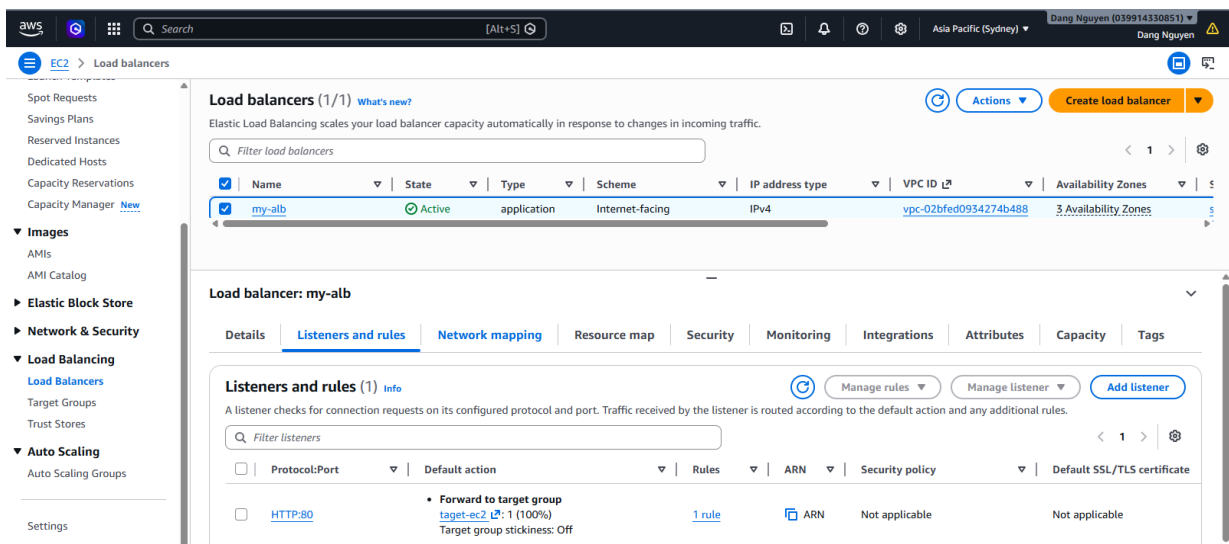
### 3.2.1 Application Load Balancer



Hình 3.2: Application Load Balancer tạo trên AWS.

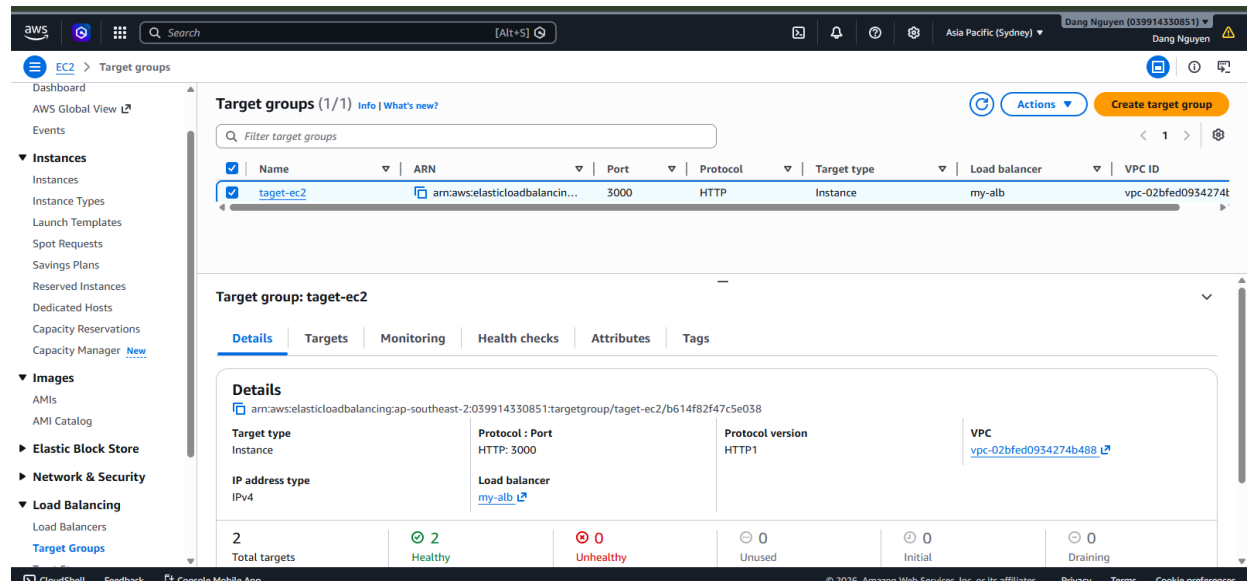
Application Load Balancer đã được tạo thành công trên AWS. Đây là thành phần tiếp nhận request từ người dùng và chuyển tiếp lưu lượng đến các EC2 Web Server phía sau thông qua Target Group.

### 3.2.2 Listener của Application Load Balancer



Hình 3.3: Listener HTTP port 80 chuyển tiếp request đến Target Group.

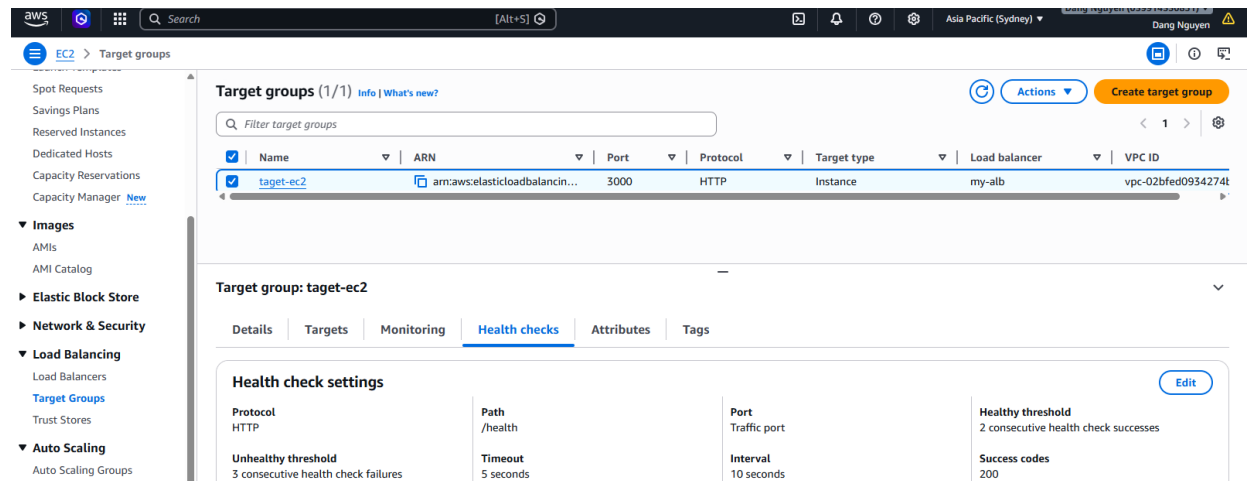
### 3.2.3 Target Group



Hình 3.4: Các EC2 trong Target Group ở trạng thái Healthy.

Các EC2 Web Server đã được thêm vào Target Group và đang ở trạng thái Healthy. Điều này chứng minh các server phía sau Load Balancer có thể nhận request và tham gia xử lý lưu lượng

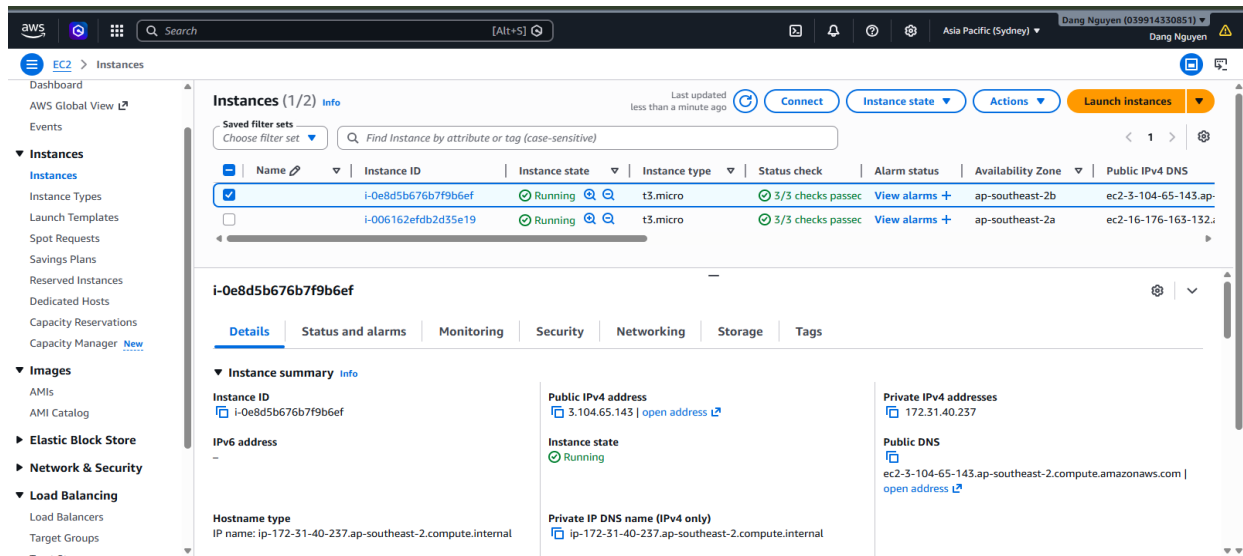
### 3.2.4 Health Check



Hình 3.5: Cấu hình Health Check của Target Group.

Health Check dùng để kiểm tra trạng thái hoạt động của các EC2. Nếu EC2 phản hồi đúng theo cấu hình, hệ thống sẽ đánh dấu Healthy và tiếp tục phân phối request đến server đó.

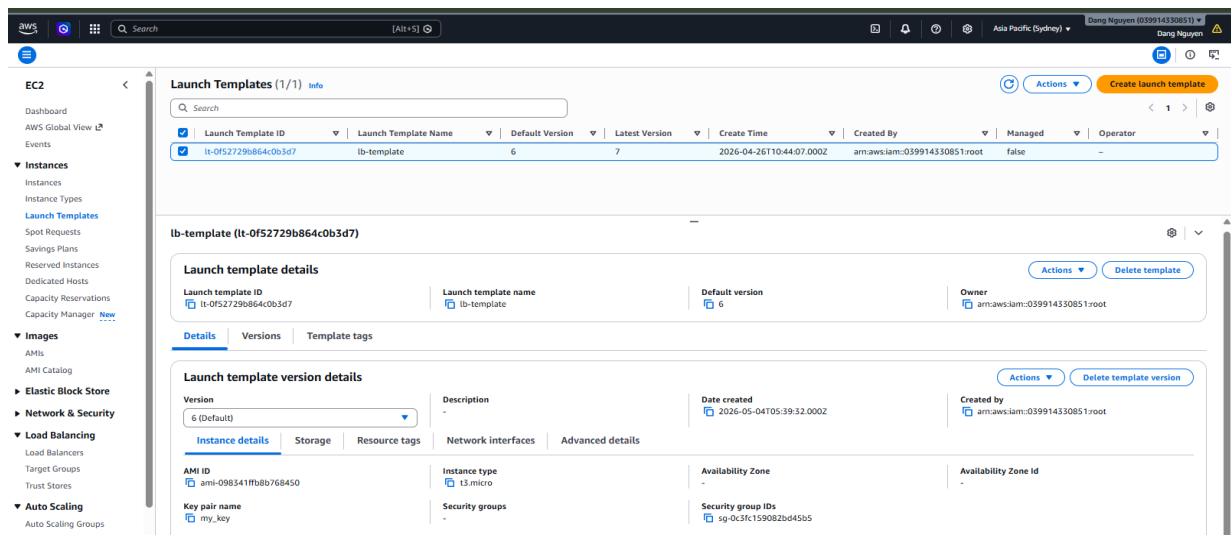
### 3.2.5 EC2 Web Server



Hình 3.6: Danh sách EC2 Web Server đang chạy trong hệ thống.

Các EC2 Web Server đang chạy trong hệ thống. Các instance này đóng vai trò xử lý request được chuyển đến từ Application Load Balancer.

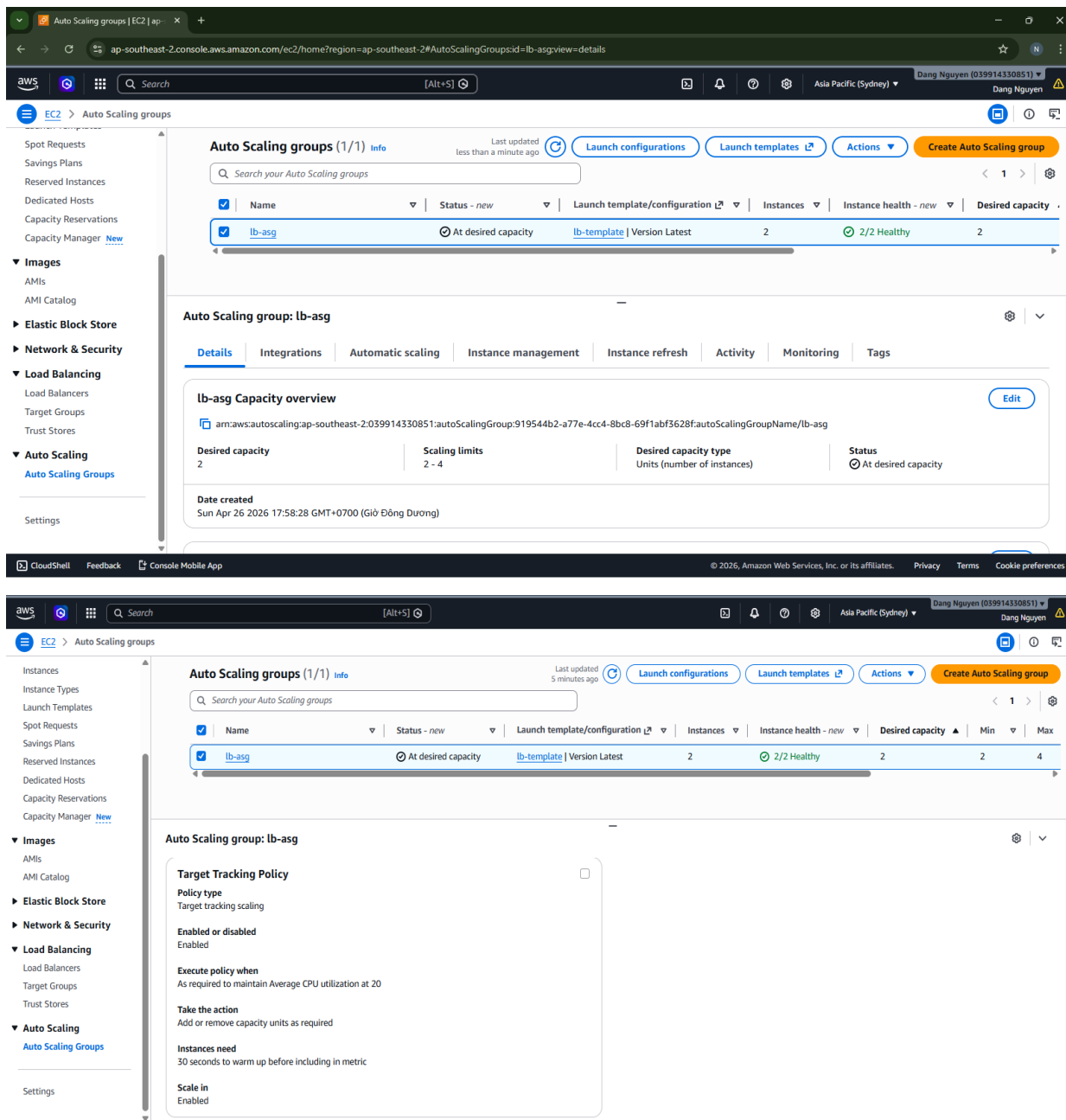
### 3.2.6 Launch Template



Hình 3.7: Launch Template khởi tạo EC2 Auto Scaling Group dựa vào CI/CD cơ bản.

Launch Template được dùng để tạo EC2 cho Auto Scaling Group. Nhờ Launch Template, các EC2 mới được tạo ra sẽ có cấu hình giống nhau và có thể nhanh chóng tham gia vào hệ thống.

### 3.2.7 Auto Scaling Group

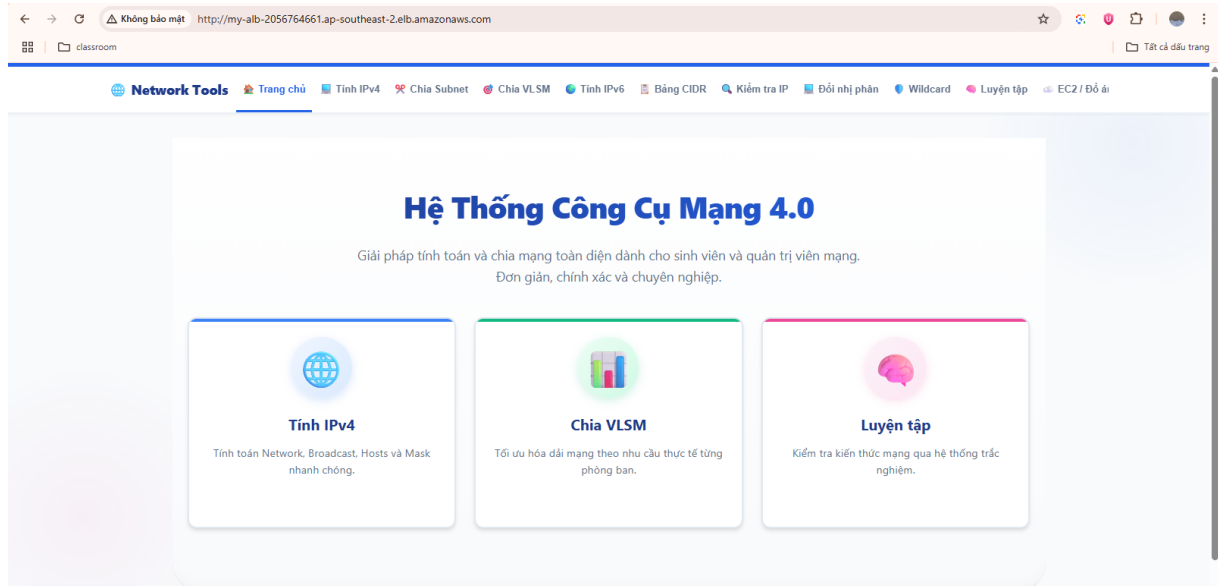


Hình 3.8: Auto Scaling Group hệ thống.

Auto Scaling Group đã được tạo và liên kết với Target Group của hệ thống. Thành phần này có nhiệm vụ quản lý số lượng EC2 Web Server theo cấu hình Min, Desired và Max. Khi tải hệ thống thay đổi, Auto Scaling Group sẽ tự động tạo thêm hoặc thu hồi EC2 để hỗ trợ cân bằng tải tốt hơn.

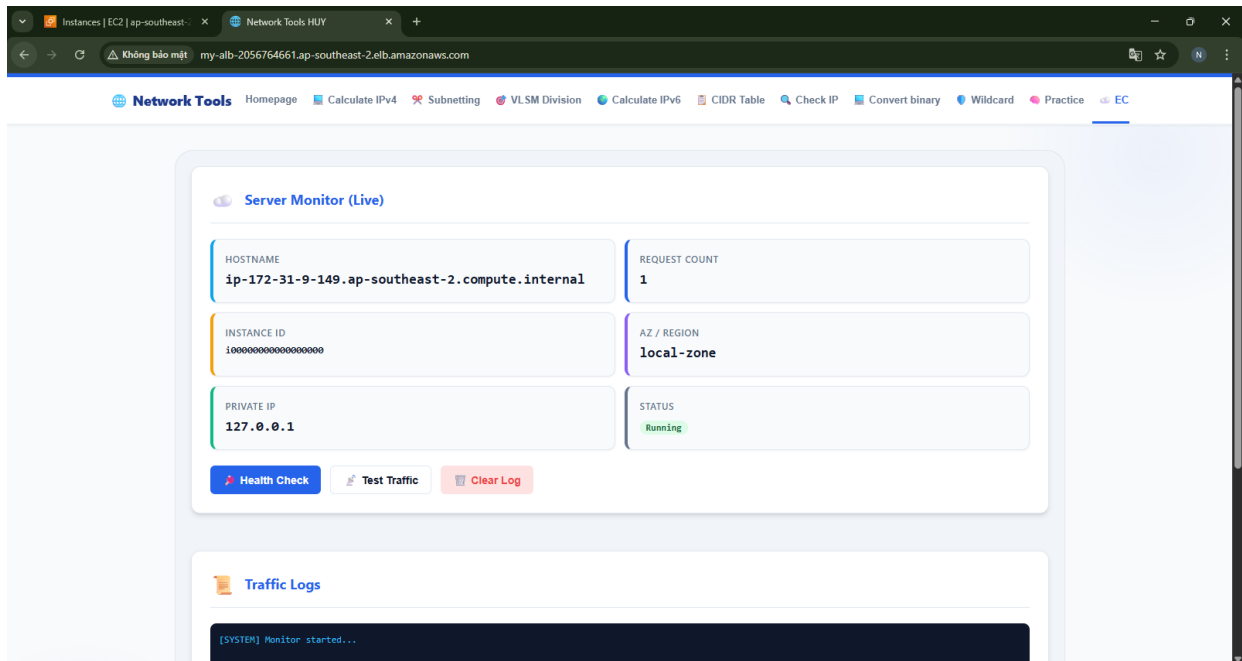
## 3.3 Kiểm tra hoạt động hệ thống

### 3.3.1 Truy cập ứng dụng thông qua DNS của ALB



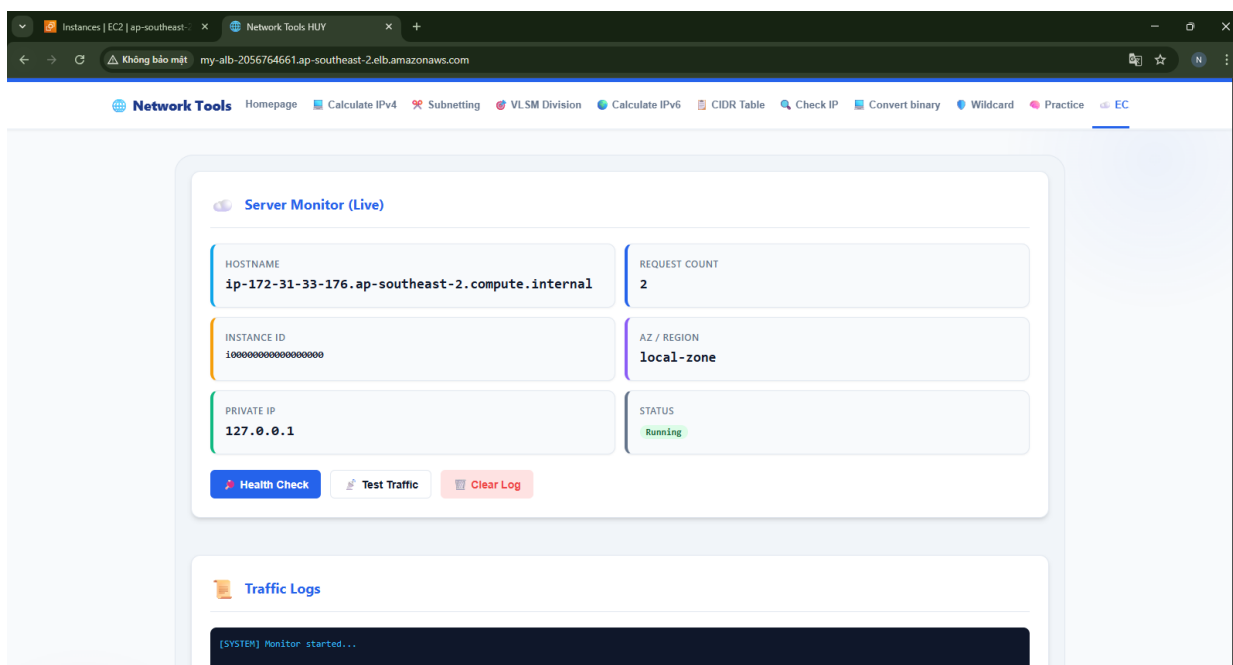
Hình 3.9: Truy cập ứng dụng thông qua DNS của Application Load Balancer.

### 3.3.2 Kiểm thử phân phối request đến các EC2



Hình 3.10: Request được phân phối đến EC2 Web Server thử nhất.





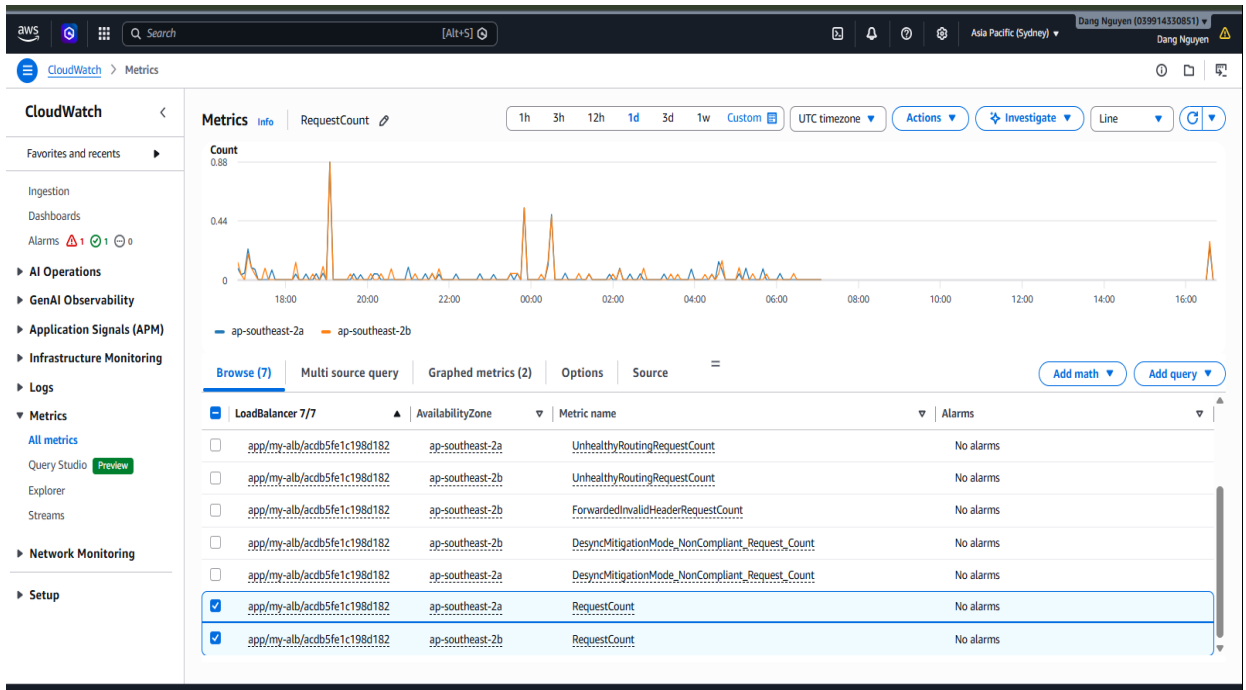
Hình 3.11: Request được phân phối đến EC2 Web Server thứ hai.

### 3.4 Giám sát và tự động mở rộng hệ thống

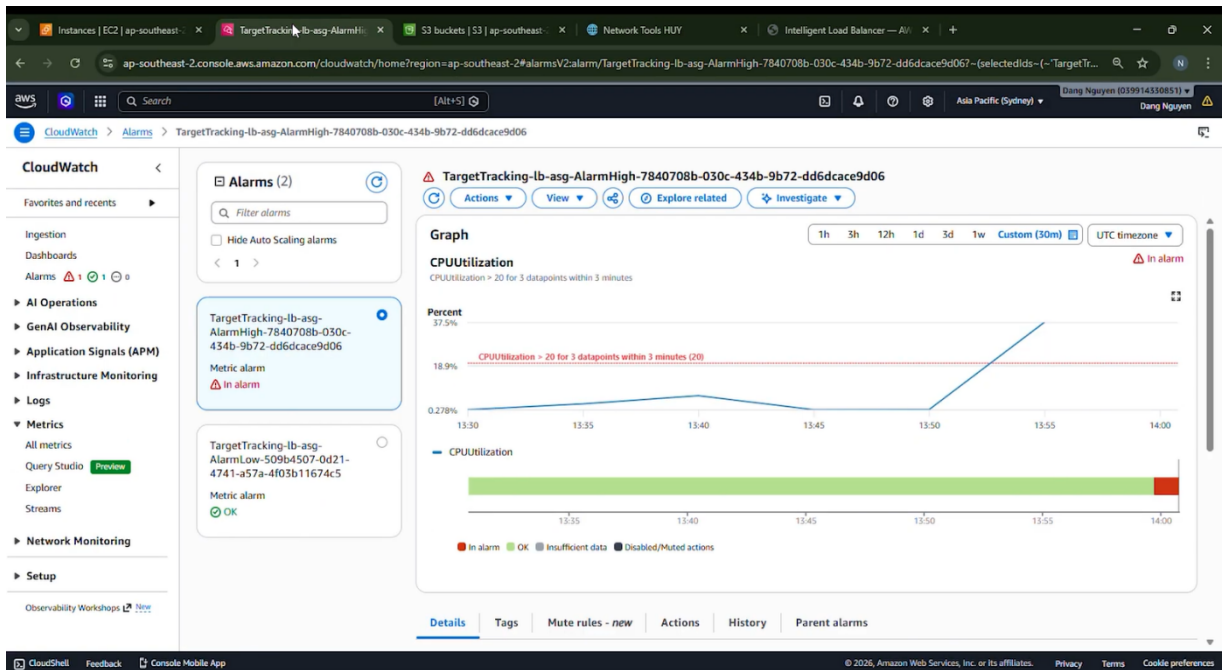
```
</body>
</html>[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$ yes > /dev/null &
[1] 27404
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$ yes > /dev/null &
[2] 27405
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$ killall yes
pkill -f yes
[2]+  Terminated                  yes > /dev/null
[1]+  Terminated                  yes > /dev/null
[ec2-user@ip-172-31-3-133 IntelligentLoadBalancer_Network]$ |
```

Hình 3.12: Giả lập ép tăng/ giảm tải CPU trên EC2 Web Server bằng lệnh Terminal.

Quá trình nhóm sử dụng lệnh Terminal để giả lập tải CPU trên EC2 Web Server. Mục đích của bước này là tạo tình huống tải tăng trong môi trường thử nghiệm, từ đó kiểm tra CloudWatch Alarm và phản ứng của Auto Scaling Group khi hệ thống vượt ngưỡng cấu hình.

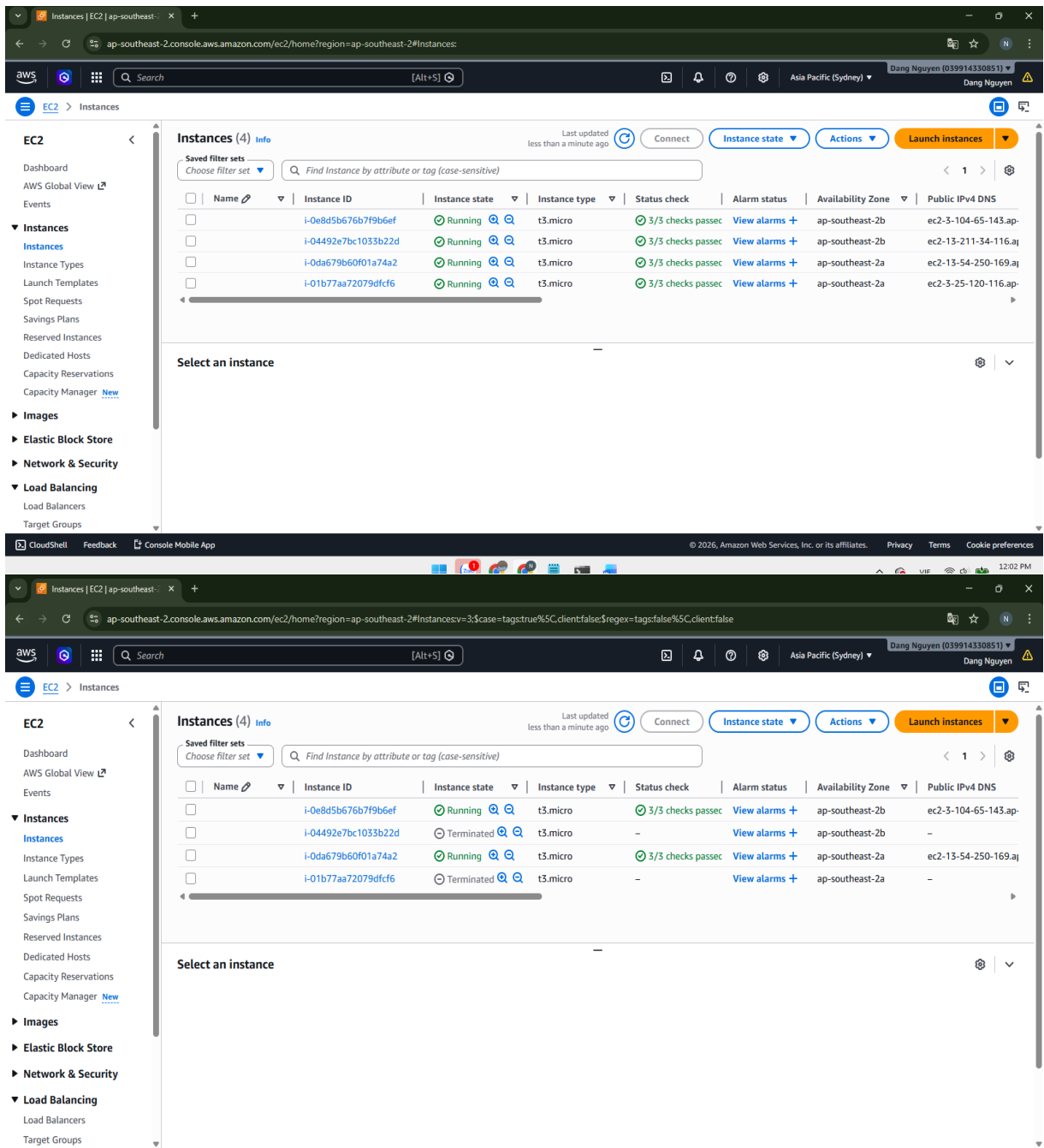


Hình 3.13: CloudWatch giám sát số lượng request của Application Load Balancer.



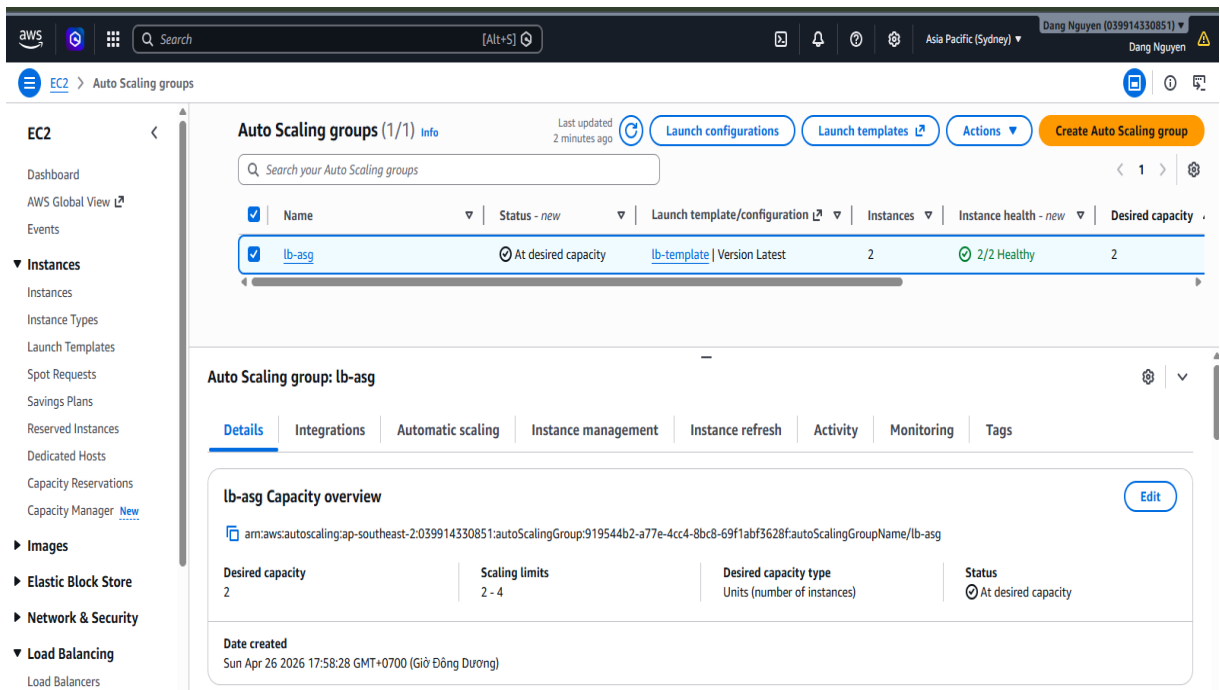
Hình 3.14: CloudWatch Alarm theo dõi ngưỡng CPU của hệ thống.

CloudWatch Alarm được cấu hình theo dõi ngưỡng CPU. Khi vượt ngưỡng đã đặt, Alarm sẽ kích hoạt Scaling Policy để Auto Scaling Group thực hiện mở rộng hệ thống.

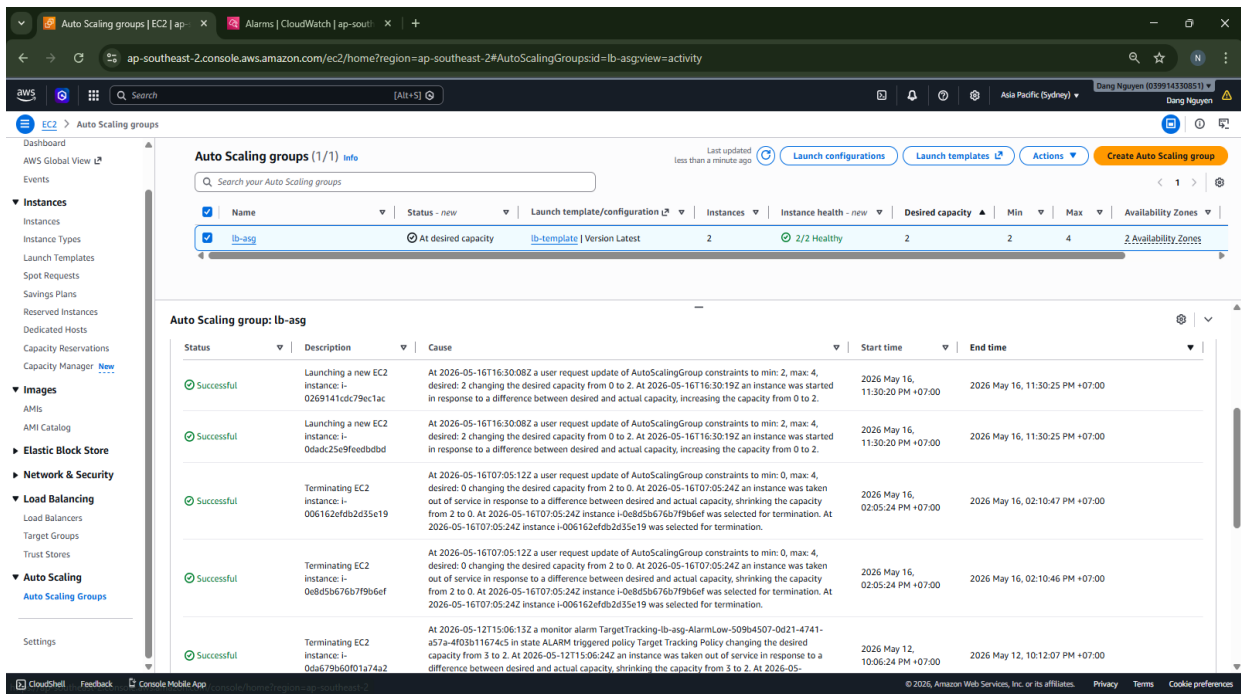


Hình 3.15: Số lượng EC2 thay đổi theo quá trình Auto Scaling.

Số lượng EC2 thay đổi trong quá trình kiểm thử Auto Scaling. Khi hệ thống vượt ngưỡng tải cấu hình, Auto Scaling Group có thể tạo thêm EC2 mới để hỗ trợ xử lý request; khi tải giảm, hệ thống sẽ thu hồi bớt EC2 để tiết kiệm tài nguyên.



Hình 3.16: Cấu hình Capacity của Auto Scaling Group trong hệ thống.

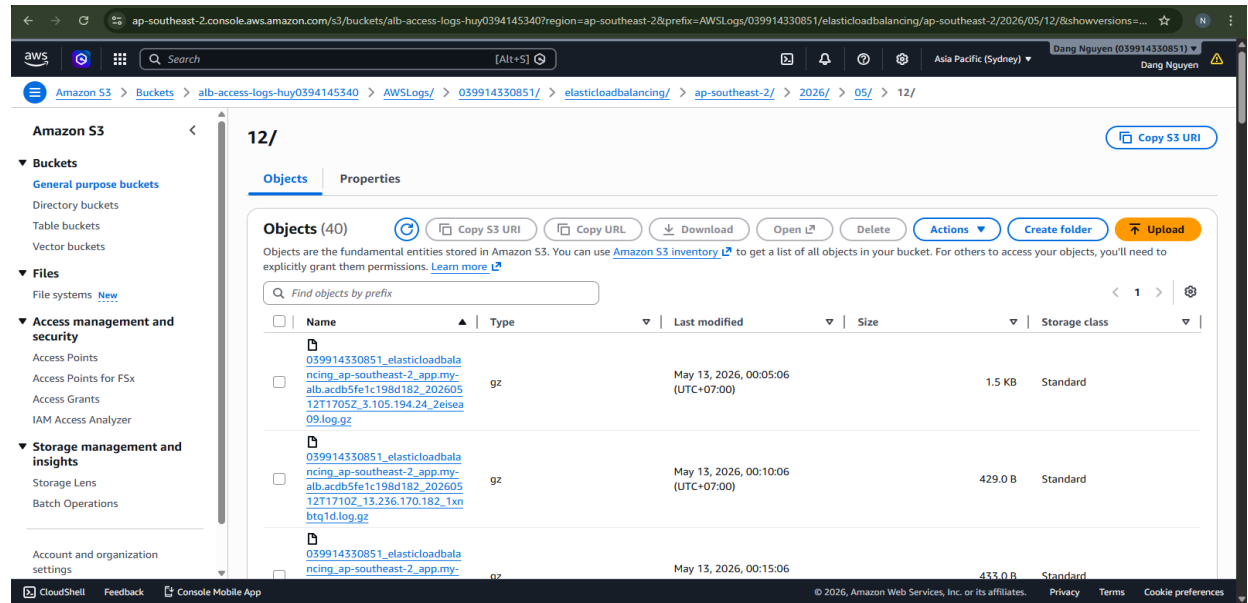


Hình 3.17: Lịch sử Auto Scaling Group tạo mới và thu hồi EC2 instance.

Ghi lại lịch sử hoạt động của Auto Scaling Group. Thông qua phần này, nhóm có thể kiểm tra các lần hệ thống tạo mới hoặc thu hồi EC2 trong quá trình kiểm thử.

## 3.5 Ghi nhận Access Log và Dashboard giám sát

### 3.5.1 Access Log trên Amazon S3



Hình 3.18: Access Log của Application Load Balancer trên Amazon S3.

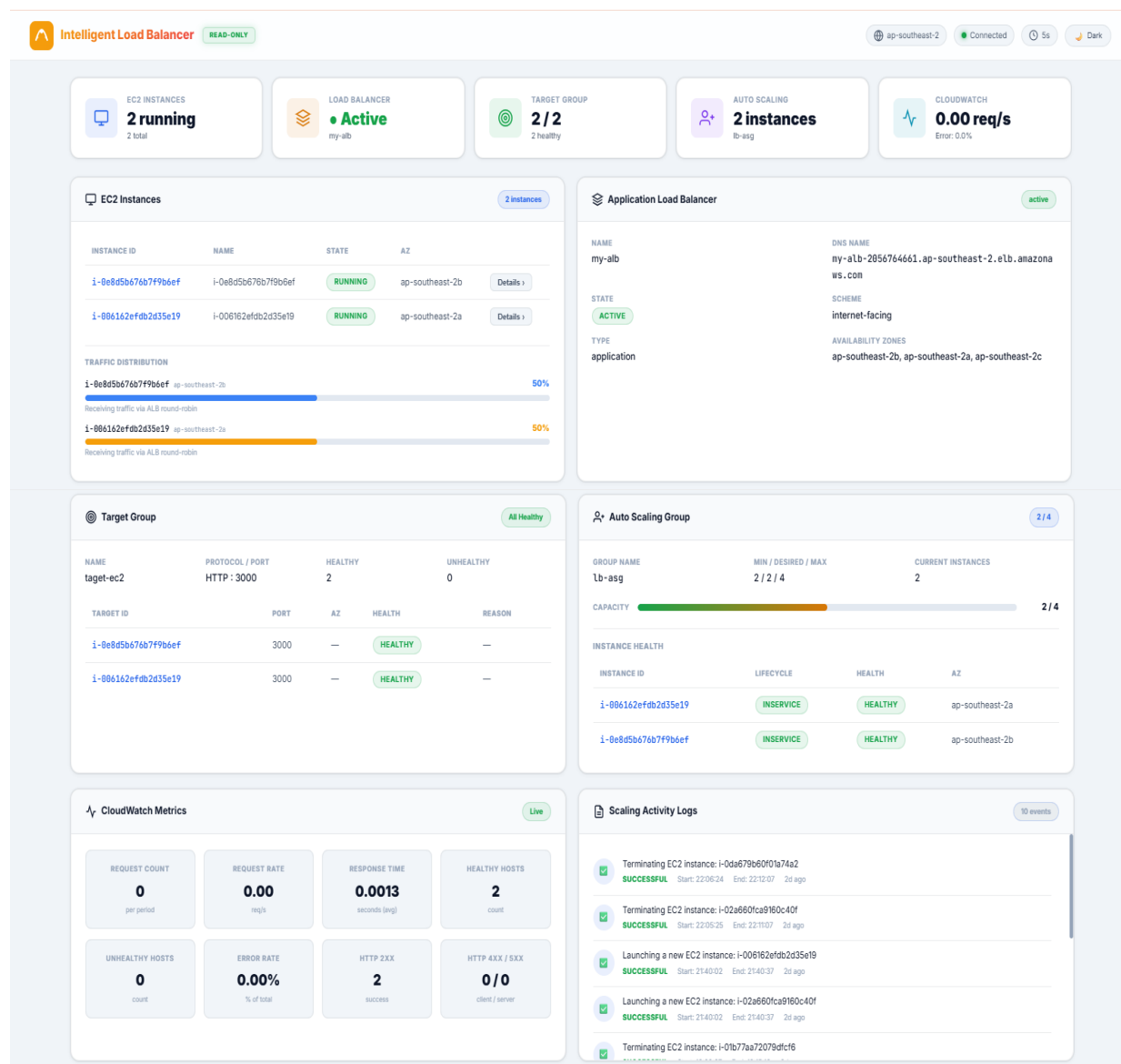
### 3.5.2 Nội dung log request



Hình 3.19: Nội dung Access Log ghi nhận request đi qua Load Balancer.

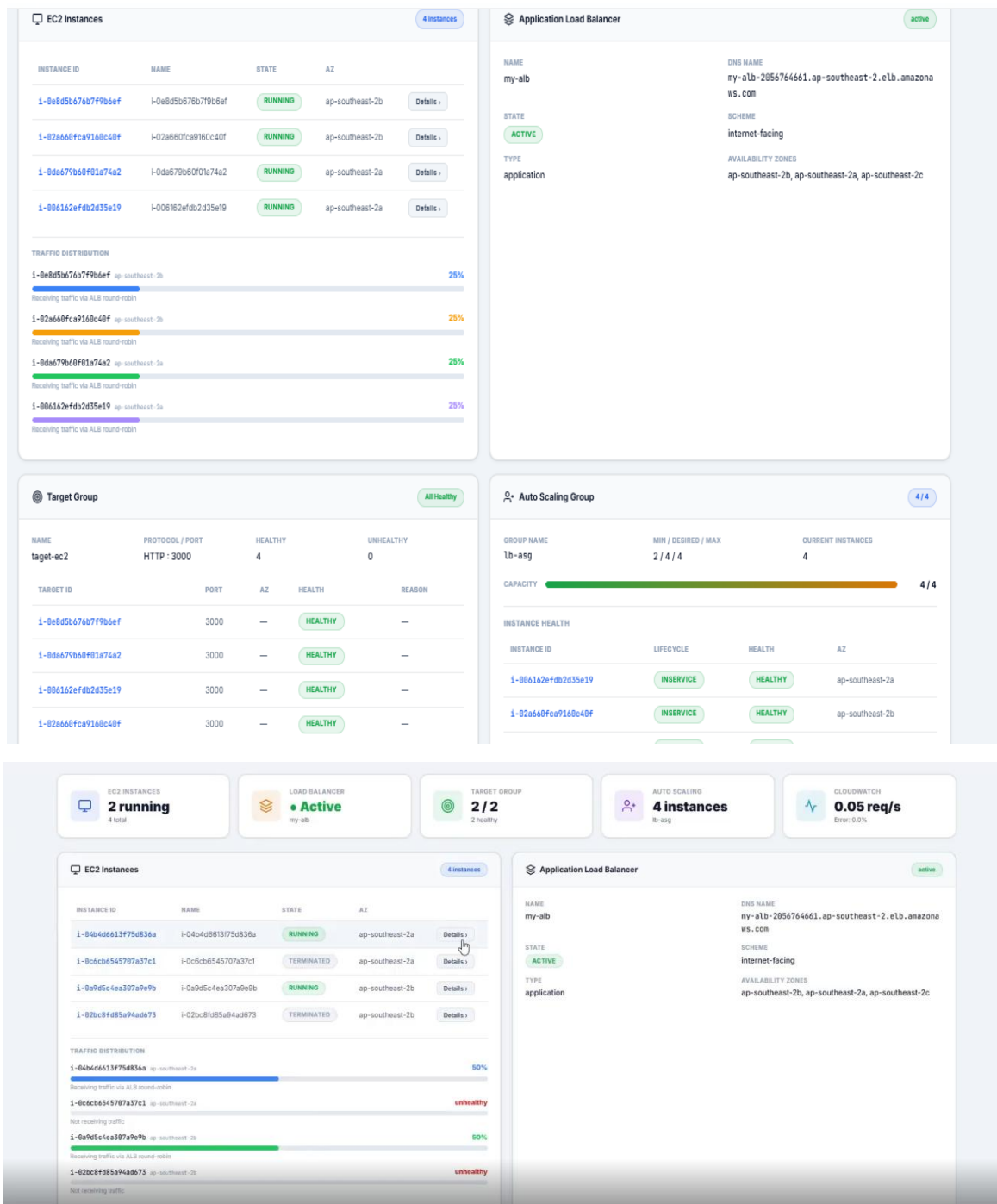
Nội dung Access Log được ghi nhận từ Application Load Balancer. Trong log có các thông tin như thời điểm request, địa chỉ truy cập, mã trạng thái phản hồi và server xử lý request. Những dữ liệu này giúp nhóm kiểm tra lại hoạt động của Load Balancer trong quá trình thử nghiệm.

### 3.5.3 Dashboard giám sát hệ thống



Hình 3.20: Dashboard liên kết AWS giám sát hệ thống.

Dashboard giám sát trạng thái tổng quan của hệ thống sau khi các thành phần chính đã được cấu hình. Thông qua Dashboard, nhóm có thể theo dõi nhanh tình trạng của Application Load Balancer, Target Group, EC2 Web Server và các chỉ số liên quan. Việc tổng hợp thông tin trên một màn hình giúp quá trình quan sát và kiểm tra hệ thống thuận tiện hơn.



Hình 3.21: Dashboard liên kết AWS giám sát hệ thống khi tăng/ giảm tải.

Dashboard trong quá trình nhóm kiểm thử tăng/giảm tải hệ thống. Khi tải thay đổi, nhóm có thể quan sát sự thay đổi của số lượng EC2, trạng thái server và hoạt động của Auto Scaling Group. Kết quả này giúp chứng minh hệ thống không chỉ phân phối request mà còn có khả năng theo dõi và phản ứng khi tải thay đổi.



## CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ

### 4.1 Kết luận

Sau quá trình tìm hiểu và triển khai đề tài Intelligent Load Balancer, nhóm đã xây dựng được một mô hình cân bằng tải cơ bản trên nền tảng AWS. Hệ thống được thiết kế với Application Load Balancer đóng vai trò tiếp nhận request từ người dùng và phân phối đến nhiều EC2 Web Server phía sau thông qua Target Group.

Thông qua quá trình thực nghiệm, nhóm nhận thấy hệ thống có khả năng phân phối request đến nhiều server khác nhau một cách ổn định. Việc này giúp giảm tải cho từng con EC2, giúp hệ thống hạn chế tình trạng quá tải khi có nhiều người truy cập cùng lúc hay vấn đề sự cố. Nhóm đã triển khai Auto Scaling để hỗ trợ mở rộng hệ thống khi cần thiết. Khi lượng truy cập tăng, hệ thống có thể tự động thêm EC2 mới để xử lý request. Điều này giúp hệ thống linh hoạt hơn và không cần can thiệp thủ công quá nhiều trong quá trình vận hành. Qua đó nhóm đã hiểu rõ hơn về cách hoạt động của Load Balancer trong thực tế, cũng như cách các dịch vụ trên AWS phối hợp với nhau để xây dựng một hệ thống có khả năng chịu tải và tự mở rộng tốt.

### 4.2 Hạn chế của đề tài

- Mô hình chỉ ở mức cân bằng tải cho Web Server, chưa triển khai cho Database.
- Việc kiểm thử chủ yếu ở mức cơ bản truy cập thủ công, chưa sử dụng các công cụ tạo tải chuyên sâu.
- Chưa tối ưu chi phí khi sử dụng tài nguyên trên AWS.

### 4.3 Hướng phát triển

- Thu thập dữ liệu hoạt động của hệ thống như CPU, RAM, số lượng request và thời gian phản hồi từ các EC2.
- Phân tích cấu trúc mạng và hành vi truy cập của người dùng để hiểu rõ tình trạng tải theo thời gian thực.
- Áp dụng AI để đề xuất kiến trúc mạng và chiến lược phân phối request tối ưu hơn, thay vì chỉ sử dụng các thuật toán cố định như Round Robin.



## TÀI LIỆU THAM KHẢO

[1] Amazon Web Services (n.d.). Elastic Load Balancing Documentation.

<https://docs.aws.amazon.com/elasticloadbalancing/>.

[Ngày truy cập: 10/05/2026].

[2] Amazon Web Services (n.d.). Amazon EC2 Documentation.

<https://docs.aws.amazon.com/ec2/>.

[Ngày truy cập: 10/05/2026].

[3] Amazon Web Services (n.d.). Amazon EC2 Auto Scaling Documentation.

<https://docs.aws.amazon.com/autoscaling/>.

[Ngày truy cập: 20/05/2026].

[4] Amazon Web Services (n.d.). Target Groups for Your Application Load Balancers.

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>.

[Ngày truy cập: 26/05/2026].